

Large Language Models for Quantum Circuit Cutting and Optimization

Zeke Laney

Department of Computer Science

University of Colorado Colorado Springs

George Mason University

United States

elaney3@gmu.edu

Michael Conner

Department of Computer Science

University of Colorado Colorado Springs

United States

mconner@uccs.edu

Armin Moin

Department of Computer Science

University of Colorado Colorado Springs

United States

amoin@uccs.edu

Abstract—Quantum circuit cutting decomposes large circuits into smaller sub-circuits executable on limited NISQ hardware, but existing methods rely on hard-coded heuristics or domain-specific training. A natural question is whether off-the-shelf large language models (LLMs), given their demonstrated circuit-analysis capabilities, can improve upon deterministic cutting strategies when embedded as orchestrators in a multi-strategy pipeline. We investigate this question through GRACE (Generative Resilient Automatic Circuit-Cutting Engine), an agentic pipeline built on LangGraph that routes circuits through an LLM-guided cutting attempt before falling back to established deterministic backends. Evaluated on 150 MQT Bench circuits across 15 families at 3–12 qubits with three seeded trials per configuration, we find that the answer is *no*: GRACE achieves a mean genuine pass rate of $45.6\% \pm 0.8$ compared to $45.1\% \pm 0.4$ for a deterministic-only baseline—a difference of +0.7 circuits that is not statistically significant (McNemar’s exact test, $p = 0.50$; Wilcoxon signed-rank, $W = 0$, $p = 0.50$), though with only 2 discordant pairs out of 450 trials the experiment has limited power to detect small effect sizes. The LLM’s first-attempt cuts pass validation on only 26.2% of circuits, with the remaining 74% recovered by deterministic fallbacks. Across all trials, 0–1 circuits per seed pass exclusively in the LLM configuration, and none through the LLM’s own cuts. Both configurations exhibit high reproducibility (97–99% identical outcomes across seeds). These results constitute a rigorous negative finding: current off-the-shelf LLMs do not improve upon deterministic circuit-cutting baselines on structured benchmark circuits at this scale. However, the experiment yields two constructive insights: (1) LLM integration is architecturally feasible without degrading output quality or reproducibility, and (2) the dominant bottleneck—exponential validation cost scaling as 6^K with cut count—is independent of strategy selection, suggesting that advances in validation efficiency, rather than cut-finding intelligence, are the binding constraint. We discuss implications for when LLM-guided compilation may become worthwhile and what would need to change.

Index Terms—gate-based quantum computing, quantum circuit cutting, artificial intelligence, machine learning, large language models, quantum software engineering

I. INTRODUCTION

Quantum computing is an emerging field of computing that promises great computational improvements over current “classical” computers. In fact, quantum computers have already demonstrated great improvement when performing a va-

riety of computations such as molecular science and chemistry [Hastings et al.(2015)] or cryptography with the help of Shor’s algorithm [Ugwisihwu et al.(2020)]. Due to these potential benefits from quantum computers, research in this field has exploded. Recent developments by companies such as IBM with their 120 qubit Nighthawk processor [IBM(2025)], Google’s low-error Willow chip [Google Quantum AI(2024)] and Amazon’s efficient Ocelot chip [Amazon Web Services(2025)] have allowed these benefits to be realized. However, despite recent advances in both qubit count and efficient error correction schemes, quantum computers are still subject to limitations in regard to the circuits they can execute.

In this work, we focus on a technique that allows large, complex quantum algorithms to be executed on current quantum computers, typically referred to as “circuit cutting” or “circuit partitioning.” This involves splitting a quantum circuit into a number of smaller “sub-circuits” and executing them in a particular manner. Because current circuit-cutting methods are heavily limited in real-world application due to restrictions such as hard-coded rules, equations, or supported Quantum Software Development Kits (QSDKs), we focus on a different approach. Dubbed Generative Resilient Automatic Circuit-Cutting Engine (GRACE), our approach focuses on the use of state-of-the-art in Artificial Intelligence (AI), specifically the Machine Learning (ML) subdiscipline, to analyze and split quantum circuits. Notably we are interested in Large Language Models (LLMs), which are built on transformer-based architectures, such as OpenAI’s GPT series and similar models. Therefore, the contribution of this paper is an empirical investigation of whether contemporary off-the-shelf LLMs can improve upon deterministic methods when embedded as orchestrators in a circuit-cutting pipeline. Our research question is: *Does LLM-guided strategy selection and cut generation improve the pass rate, efficiency, or generality of quantum circuit cutting compared to a deterministic-only baseline?* As we report in Section V, the answer on the current benchmark is negative: the LLM achieves statistical parity with the baseline but does not exceed it. We frame this as a rigorous negative result with architectural lessons, identifying the conditions under which LLM-guided cutting might become beneficial and the bottlenecks that currently prevent it.

The remainder of this paper is structured as follows. Section II provides background information regarding quantum computing. To illustrate the current state of quantum circuit cutting, Section III reviews recent publications in this field. We propose our novel approach in Section IV and the experimental study in Section V validates, evaluates and discusses the approach. Section VI discusses threats to validity, and Section VII concludes with future work.

II. BACKGROUND

Current quantum hardware is limited in qubit count and coherence time, restricting the size of circuits that can be executed directly. Circuit cutting addresses this by partitioning a large quantum circuit into smaller sub-circuits that run independently, whose outputs are classically recombined to reproduce the original result. The key challenge is that classical post-processing scales exponentially with the number of cuts: cutting a single CNOT or CZ gate via quasi-probability decomposition (QPD) incurs a $\kappa^2 = 9 \times$ sampling overhead, and K simultaneous cuts scale as 9^K . For a one-day execution budget, a superconducting machine sampling at 1 kHz can accommodate roughly 4 cuts, rising to about 7 at 1 MHz [Brandhofer et al.(2023)]. Minimizing the number and placement of cuts is therefore critical, and motivates the search for intelligent cut-finding strategies.

III. RELATED WORKS

Several approaches to quantum circuit cutting have been developed, spanning classical optimization, domain-specific techniques, and early explorations of large language models.

Peng et al. introduced a tensor-network-based cluster simulation scheme that identifies cuts from circuit decomposability, demonstrating feasibility but incurring significant sampling overhead from repeated subcircuit executions [Peng et al.(2020)]. Shehzad et al. advanced cut-finding with an automated best-first search that minimizes sampling overhead given a maximum subcircuit size, with user-defined constraints enabling early termination via greedy approximation [Shehzad et al.(2024)]. Their `find_cuts` function is incorporated as one of GRACE’s cutting backends, wrapped within an agentic pipeline that adds multi-strategy selection and validation. In the domain-specific space, Sadhu et al. proposed CutQAS, a multi-agent reinforcement learning system for quantum chemistry simulations [Sadhu et al.(2025)], but its agents are trained specifically for molecular simulation and would require retraining for other problem classes.

The most directly related prior work is by Sinha et al., who explored using LLMs for circuit partitioning by partially retraining models such as Llama 3.1 and Mistral on QASM code [Sinha et al.(2025)]. This fine-tuning yielded a partition label accuracy of 53.4%, compared to 0% from off-the-shelf models using few-shot prompting. Notably, their metric evaluates whether the model reproduces a specific target partition assignment, not whether the resulting decomposition passes end-to-end equivalence validation as in the present work; the two figures are therefore not directly comparable. However,

this approach treats the LLM as a monolithic partitioner requiring extensive fine-tuning. Subsequent work has shown that LLMs demonstrate strong circuit analysis capabilities without task-specific pretraining [d’Aloisio et al.(2024)], suggesting that fine-tuning may not be the most efficient use of these models.

These approaches reveal a consistent gap: classical methods provide rigorous optimization but lack adaptive strategy selection, domain-specific methods do not generalize, and existing LLM-based work requires costly fine-tuning while foregoing access to established cutting tools. GRACE addresses this gap by using an off-the-shelf LLM not as a direct partitioner, but as an agentic orchestrator within a multi-strategy pipeline, equipping it with access to multiple cutting backends and delegating strategy selection, parameter tuning, and result validation to a structured LangGraph workflow.

IV. PROPOSED APPROACH

The approach proposed in this paper is an LLM-driven agentic workflow that combines large-language-model reasoning with external, deterministic tooling to perform quantum circuit cutting. The system, which we call GRACE, is implemented in Python using LangGraph, an open-source framework well suited to building stateful, multi-step agentic systems. The workflow is expressed as a directed graph of seven nodes connected by a mixture of unconditional (hard) and conditional edges, shown in Fig. 1 and described below. A central design feature is the retry loop: when a chosen strategy fails validation, control returns to the router, which is explicitly told which strategies have already failed so that it never re-selects them.

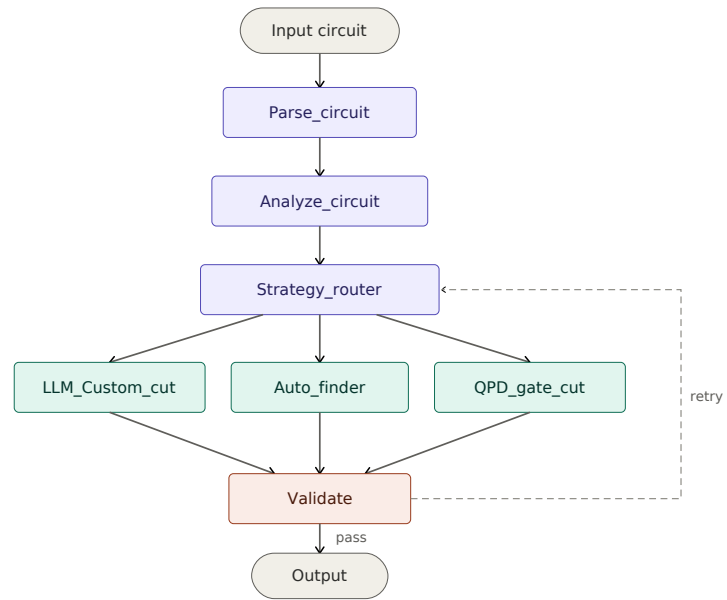


Fig. 1. The GRACE LangGraph pipeline architecture.

A. Node Architecture

- 1) **Parse_circuit** – Takes an OpenQASM circuit as input, loads it into a Qiskit `QuantumCircuit`, and produces a concise summary of the circuit’s structurally relevant features for downstream nodes.
- 2) **Analyze_circuit** – Using the parsed circuit and its summary, this node computes quantitative metrics (qubit count, depth, two-qubit gate count, connectivity density, non-Clifford count, and dense/deep flags) and prompts the LLM to produce a short descriptive analysis for later nodes.
- 3) **Strategy_router** – On the first attempt of every run, the router unconditionally selects `LLM_Custom_cut` (Node 5) so that the LLM-driven strategy is always tried first. Only when that initial attempt fails validation does the router consult the LLM to select among the remaining two state-of-the-art strategies (Nodes 4 and 6), using the analysis, the circuit metrics, and the list of strategies that have already failed. If the LLM call is unavailable or returns an unmappable reply, the router falls back to a deterministic priority ordering rather than failing.
- 4) **Auto_finder** – Invokes `find_cuts()` from the Qiskit circuit-cutting addon to automatically discover cut locations and partition the circuit.
- 5) **LLM_Custom_cut** – Prompts the LLM to propose a bespoke cutting plan, then executes it through the same underlying machinery as Node 6.
- 6) **QPD_gate_cut** – Applies QPD gate cutting from the Qiskit addon to selected cross-partition two-qubit gates.
- 7) **Validate** – Checks whether valid sub-circuits were produced. On success the graph terminates; on failure the offending strategy is recorded in the run state and control routes back to `Strategy_router` for another attempt.

An earlier `QPD_wire_cut` node was removed after benchmarking showed its temporal-centrality heuristic consistently produced trivial single-subcircuit results, failing validation on every circuit.

B. LLM Configuration

LLM calls are routed through OpenRouter using an OpenAI-compatible client. The router node queries the `deepseek/deepseek-v4-flash` model with temperature set to 0 and a 1200-token output budget; explicit reasoning (“thinking”) mode is disabled to keep replies terse and to avoid exhausting the token budget on hidden chain-of-thought.¹ The system prompt instructs the model to select exactly one strategy in a strict two-line format (`STRATEGY / REASON`), suppressing verbose output so the reply can be parsed deterministically. When strategies have already failed, the prompt is augmented to forbid their re-selection.

¹Earlier development used a free-tier `gpt-oss-120b` endpoint; the model was switched to DeepSeek-V4 Flash for reliability. Because different backends can produce different routing decisions, all reported benchmark results were generated with a single fixed model.

C. Strategy Selection Policy

Every run begins with `LLM_Custom_cut` as the mandatory first attempt; the LLM-based strategy selector described above is consulted only on retries after the initial attempt has failed validation. When the selector is invoked, its system prompt is paired with a user prompt that supplies the router with (i) the list of previously failed strategies, (ii) a one-line description of each available strategy, and (iii) the circuit summary, analysis, and key metrics. Each strategy is paired with a one-line description of when it is most appropriate (e.g., `qpd_gate_cut` for circuits with few cross-partition two-qubit gates), enabling the router to match the circuit to the best-suited technique. As a deterministic backstop, if the LLM re-selects a strategy that has already failed, the router overrides the choice with the highest-priority untried strategy from a fixed ordering (`qpd_gate_cut`, `auto_finder`).

D. Determinism and Seed Control

Two sources of run-to-run nondeterminism affect the pipeline. The first is `find_cuts()`, whose best-first search over cutting actions is stochastic when invoked without an explicit seed; the second is the LLM backend, which does not guarantee bitwise-identical outputs across calls even at temperature 0.

To make comparisons reproducible, GRACE threads a single run-level seed through every stochastic component. The seed is supplied to the batch harness with a `--seed` argument and is (i) written into `AutoFinderConfig` so that `find_cuts()` explores cut configurations deterministically, (ii) used to seed Python’s `random` module and NumPy’s global generator before the graph is constructed, and (iii) recorded in each run’s exported metadata alongside the strategy trace, so that any individual result can be reproduced in isolation. The seed is also embedded in the per-run output directory name, allowing multiple seeded replicates of the same circuit corpus to coexist without collision.

Fixing the seed makes a single configuration reproducible but does not by itself characterize the variance of the pipeline. We therefore treat the seed as an experimental factor and repeat the full corpus under three seeds (101, 202, 303), reporting both per-seed results and cross-seed means with standard deviations in Section V.

E. Output Handling and Equivalence Validation

When the graph terminates successfully, the original circuit, the marked and cut circuits, the generated sub-circuits, a human-readable overview, and structured metadata (including the per-gate QPD bases, κ values, and total sampling overhead) are exported to a dedicated per-run directory for inspection and offline validation.

Correctness is established by three equivalence checkers (one per cutting node), each of which *re-derives* the QPD decomposition live from the exported circuit to avoid round-trip serialization hazards. Validation proceeds through structural, channel, and reconstruction checks, the last of which recombines sub-experiment results into expectation values and

compares them against exact statevector values. All three checkers apply a single, unified pass criterion:

$$\max_O |\langle O \rangle_{\text{reconstructed}} - \langle O \rangle_{\text{exact}}| \leq \epsilon_{\text{base}} + z \sigma_{\text{shot}},$$

where ϵ_{base} is a fixed systematic-error budget, $\sigma_{\text{shot}} = \sqrt{\sum_i c_i^2 / N_{\text{shots}}}$ is the statistical shot-noise bound of the QPD estimator (with c_i the quasi-probability coefficients and N_{shots} the per-sub-experiment sample count), and z is the number of standard deviations included in the margin. This coefficient-aware tolerance replaces a fixed threshold: a flat number would judge a one-cut run (overhead 9) and a four-cut run (overhead 6561) by wildly different statistical standards, either producing spurious failures on valid many-cut runs or being uselessly loose on few-cut runs. Agreement within this tolerance yields a `PASS`; a mismatch yields `FAIL_EQUIVALENCE`. Expectation values are reconstructed with Qiskit Aer’s `SamplerV2` (required for mid-circuit measurements in QPD sub-experiments), and checkers default to exact mode, enumerating all QPD terms with their exact quasi-probability weights so that a reported `PASS` reflects the decomposition itself rather than a lucky sample.

To evaluate GRACE at scale, circuits are fetched from the Munich Quantum Toolkit (MQT) Benchmark Library as OpenQASM 2.0 files and each is run through the pipeline in an isolated subprocess to prevent single-circuit failures from halting the batch.

Before the (expensive) offline equivalence check is launched, an adaptive feasibility gate estimates the wall-clock cost using a fitted model:

$$\hat{t} = 6^K \cdot K_0 \cdot n_{\text{obs}} \cdot G^{n_{\text{qubits}}},$$

where K is the number of cuts, n_{obs} is the observable-battery size (reproduced per-strategy as the validators build it), and K_0 and G are constants fitted by least squares on the logarithmic residuals of prior validation runs. The 6^K factor reflects the exponential growth in QPD variants: cutting a single CX/CZ gate incurs an overhead factor $\gamma^2 = \kappa^2 = 9$, and K simultaneous cuts scale as γ^{2K} , so classical overhead explodes well before the cut count becomes large [Schmitt et al.(2025)]. A run is recorded as `VALIDATION_INFEASIBLE` when $\hat{t}$ exceeds a configurable time budget; a secondary cap on the total sampling overhead independently guards against edge cases that pass the cost model yet still exhaust available compute. When a strategy is ruled infeasible, control reroutes to a leaner strategy (typically `qpd_gate_cut`) whenever an untried option remains; only when no leaner option is available is a terminal `VALIDATION_INFEASIBLE` conceded. Feasible runs are dispatched to the appropriate equivalence checker under a wall-clock timeout, and every outcome is logged with a structured status.² Runs are resumable: the harness skips circuits that have already reached a terminal status on restart.

²Possible statuses: `PASS`, `PASS_NO_CUT`, `FAIL_EQUIVALENCE`, `PIPELINE_VALIDATION_FAILED`, `PIPELINE_ERROR`, `VALIDATOR_ERROR`, `TIMEOUT_PIPELINE`, `TIMEOUT_VALIDATOR`, `VALIDATION_INFEASIBLE`, `SKIPPED_TOO_LARGE`.

V. EXPERIMENTAL RESULTS

We evaluate GRACE on 150 circuits drawn from the MQT Bench library, spanning 15 circuit families (10 circuits each, from 3 to 12 qubits). To characterize variance, we run three seeded trials per configuration (seeds 101, 202, 303), yielding six total batch runs. We compare two configurations: (1) a *no-LLM baseline*, in which the `GRACE_NO_LLM` environment variable disables the mandatory `llm_custom_cut` first-attempt policy and routes circuits directly to deterministic strategies (`qpd_gate_cut` $\rightarrow$ `auto_finder`); and (2) *LLM-enabled GRACE*, the full pipeline with `llm_custom_cut` as the mandatory first strategy. Both configurations share identical validation logic, cost-model feasibility gates, fallback chains, and per-seed `AutoFinderConfig` seeds. The LLM backend is DeepSeek-V4-Flash via OpenRouter.

A circuit is counted as a *genuine pass* if validation succeeds on a cutting decomposition (i.e., the reconstructed distribution matches the original within tolerance). Circuits classified as `PASS_NO_CUT`—where `auto_finder` determines no cut is needed—are reported separately and excluded from the pass rate.

A. Aggregate Comparison

Table I summarizes pass rates across all three seeds. The no-LLM baseline achieves a mean genuine pass rate of 45.1% (67.7 ± 0.6 circuits), while LLM-enabled GRACE achieves 45.6% (68.3 ± 1.2 circuits)—a mean delta of +0.7 circuits. To quantify significance, we applied two complementary tests to per-circuit pass/fail outcomes paired across the 450 circuit-trials (150 circuits $\times$ 3 seeds). A Wilcoxon signed-rank test on the paired differences yielded $W = 0$, $p = 0.50$, with only 2 of 450 pairs producing nonzero differences (both favoring the LLM configuration). McNemar’s exact test on the 2×2 discordance table (203 both-pass, 2 LLM-only, 0 no-LLM-only, 245 both-fail) confirmed the result ($p = 0.50$). Neither test rejects the null hypothesis of equal pass rates. However, the near-total concordance that makes the result unsurprising also implies very low statistical power: with only 2 discordant pairs, the experiment could not detect an effect smaller than roughly 3–5 circuits per trial at $\alpha = 0.05$. We therefore interpret this as *consistent with* no difference rather than as strong evidence of equivalence; the latter would require a formal equivalence test (e.g., TOST) with a larger number of independent replicates.

TABLE I
AGGREGATE GENUINE PASS RATES ACROSS THREE SEEDED TRIALS.

Config	Seed 101	Seed 202	Seed 303	Mean	σ
LLM-enabled	67 (44.7%)	69 (46.0%)	69 (46.0%)	68.3	1.15
No-LLM	67 (44.7%)	68 (45.3%)	68 (45.3%)	67.7	0.58
Δ	0	+1	+1	+0.7	—

B. LLM First-Attempt Precision

Because the pipeline forces `llm_custom_cut` as the first strategy on all 150 circuits, the JSONL summary logs—which record only the *final* winning strategy—create an appearance of high LLM precision through survivorship bias: only circuits where the LLM succeeded retain the `llm_custom_cut` label. Table II reports the true first-attempt precision by tracking which circuits required rerouting to a deterministic fallback.

TABLE II
LLM FIRST-ATTEMPT OUTCOMES ACROSS 150 CIRCUITS PER SEED.

Outcome	S101	S202	S303	Mean
LLM cut passed validation	38	42	38	39.3 (26.2%)
Rerouted (LLM failed)	110	108	111	109.7 (73.1%)
→ fallback PASS	29	27	31	29.0
→ V_INFEASIBLE	57	55	56	56.0
→ PVF	13	15	14	14.0
→ PASS_NO_CUT	11	11	10	10.7
→ TIMEOUT	2	0	1	1.0
Remaining (LLM-direct PASS_NO_CUT)	2	0	1	1.0

The LLM produces a validation-passing cut on only $\sim 26\%$ of circuits. The remaining $\sim 74\%$ are recovered by deterministic fallbacks through the rerouting mechanism. Without the fallback chain, the LLM-only mode would pass only ~ 39 circuits versus the baseline’s ~ 68 , confirming that the rerouting mechanism is essential to the pipeline’s viability.

C. LLM First-Attempt Success by Circuit Family

Table III disaggregates LLM first-attempt success by circuit family, revealing a clear pattern: the LLM excels on structurally simple circuits with few two-qubit gates and regular topology, precisely the families where deterministic methods already achieve high pass rates.

TABLE III
MEAN LLM FIRST-ATTEMPT SUCCESSES PER FAMILY (OUT OF 10).

Family	Mean successes	Notes
ghz	9.0	Best LLM family
bv	8.0	Simple structure
dj	7.7	Simple structure
qnn	7.3	Strong LLM performance
graphstate	2.0	Low despite 100% final rate
qpeexact	2.0	Mostly fallback-solved
wstate	1.3	Mostly fallback-solved
qaoa	1.0	Almost entirely fallback
qft	1.0	Almost entirely fallback
ae-vqe	0.0	Never succeeds

D. Per-Family Final Pass Rates

Table IV presents final pass rates averaged over the three seeds. Of 15 circuit families, 13 show identical pass rates across both configurations. The two exceptions—`ae` and `wstate`—differ by a mean of 0.3 circuits, consistent with noise rather than a systematic effect. The pass/fail boundary is driven by circuit structure and the 6^K validation cost, not by strategy selection.

TABLE IV
PER-FAMILY FINAL PASS RATES (MEAN OF 3 SEEDS, % OF 10 CIRCUITS).

Family	LLM (%)	No-LLM (%)	Δ
bv, dj, ghz, graphstate, qnn	100	100	0
wstate	90 ± 10	86.7 ± 5.8	+0.3
qaoa	40	40	0
qft	20	20	0
qpeexact	20	20	0
qftentangled	10	10	0
ae	3.3 ± 5.8	0	+0.3
grover, qwalk, randomcircuit, vqe_two_local	0	0	0

E. Venn Overlap

Table V examines per-circuit outcome differences. Across all three seeds, the two configurations produce near-perfect overlap: 0–1 circuits per trial pass exclusively in the LLM configuration, and none pass exclusively in the no-LLM configuration.

TABLE V
PER-SEED VENN OVERLAP BETWEEN CONFIGURATIONS.

Seed	Both	LLM-only	Circuit	Winning strat.	
101	67	0	—	—	No
202	68	1	wstate_n12	qpd_gate_cut	
303	68	1	ae_n3	auto_finder	

circuits pass exclusively in the no-LLM configuration on any seed. Both LLM-only passes are produced by deterministic fallback strategies, not by the LLM’s own cuts.

Notably, neither LLM-only pass is produced by `llm_custom_cut` itself. In the seed 202 case, `wstate_n12` passes because the rerouting cascade lands on `qpd_gate_cut`, which finds a 2-cut solution ($\text{OH} = 81$), while the no-LLM pipeline’s `auto_finder` proposes 4 cuts whose $6^4 = 1296$ sub-experiments exceed the validation budget. The seed 303 `ae_n3` case records as `auto_finder` with overhead 1.0 and may reflect a tagging anomaly between `PASS` and `PASS_NO_CUT`. No circuit in the entire corpus is solved exclusively by the LLM’s cut when deterministic strategies cannot solve it.

F. Failure Mode Distribution

Table VI aggregates failure modes across all three seeds (450 circuit-trials per configuration). Total failures are nearly identical (213 vs. 214), but the LLM mode shifts their composition: fewer `VALIDATION_INFEASIBLE` outcomes (the LLM sometimes finds fewer cuts) are offset by more `PIPELINE_VALIDATION_FAILED` outcomes (the rerouting cascade pushes circuits through strategies that produce structurally valid but semantically incorrect cuts, caught by the equivalence validator).

G. Timing

Table VII reports per-circuit timing for passing circuits. The LLM pipeline is $1.4\text{--}3\times$ slower at the median, driven by two factors: LLM API call latency on the mandatory first attempt,

TABLE VI
FAILURE MODE DISTRIBUTION (SUMMED ACROSS 3 SEEDS, 450 TRIALS EACH).

Failure mode	LLM	No-LLM	Δ
VALIDATION_INFEASIBLE	168	193	-25
PIPELINE_VALIDATION_FAILED	42	17	+25
TIMEOUT_PIPELINE	3	4	-1
Total failures	213	214	-1

and the cost of rerouted passes that pay for both the failed LLM attempt and the subsequent fallback.

TABLE VII
PER-CIRCUIT MEDIAN EXECUTION TIME (PASS CIRCUITS ONLY).

Seed	LLM (s)	No-LLM (s)	Ratio
101	62.8	45.9	1.37×
202	34.1	11.5	2.97×
303	40.5	23.0	1.76×

The overhead is concentrated in rerouted passes (2–3× slower, paying for both attempts), while LLM-direct passes are only 1.5–2× slower. Total wall-clock time across all 150 circuits averages 6.3 h for LLM-enabled versus 6.0 h for the baseline (1.08×).

H. Reproducibility

Table VIII reports cross-seed reproducibility. Both configurations are highly stable: 96.7% of circuits produce identical outcomes across all three seeds in LLM mode, and 98.7% in no-LLM mode. The LLM adds slight instability (5 vs. 2 unstable circuits), concentrated in the wstate family (borderline validation cost) and grover (timeout-boundary behavior).

TABLE VIII
CROSS-SEED REPRODUCIBILITY (150 CIRCUITS, 3 SEEDS).

Agreement level	LLM	No-LLM
All 3 seeds identical	145 (96.7%)	148 (98.7%)
2-of-3 agree	5	2
All differ	0	0

I. Discussion

Our results are a negative finding: the LLM does not improve upon the deterministic baseline on the MQT Bench corpus at 3–12 qubits. GRACE achieves statistical parity across all three seeded trials, establishing that integration is feasible without degradation, but the LLM contributes no measurable advantage in pass rate, efficiency, or coverage.

Pass rates decrease monotonically from 68.9% at 3 qubits to 35.6% at 12 qubits, with the LLM showing a marginal advantage only at the extremes (+2.2 and +2.3 percentage points at 3 and 12 qubits) and zero difference for 4–11 qubits. In terms of strategy attribution, `auto_finder` dominates in LLM mode (81–82 circuits per seed) as the last fallback,

while `qpd_gate_cut` is primary in no-LLM mode (80–82 circuits).

Four principal findings emerge. First, the pass-rate difference of +0.7 circuits (68.3 ± 1.2 vs. 67.7 ± 0.6) is not statistically significant (Wilcoxon $p = 0.50$, McNemar $p = 0.50$), though the experiment’s low power (2 discordant pairs out of 450) limits the strength of this conclusion. Second, the LLM’s first-attempt precision of $\sim 26\%$ means that nearly three-quarters of all circuits require deterministic fallback, making the rerouting mechanism essential rather than supplementary. Third, the near-perfect Venn overlap (0–1 exclusive passes per trial, none from the LLM’s own cuts) is the strongest negative result: no circuit in the corpus is solved by the LLM that deterministic strategies cannot also solve. Fourth, the failure mode shift—25 fewer `VALIDATION_INFEASIBLE` offset by 25 more `PIPELINE_VALIDATION_FAILED`—shows that the LLM does find lower-cut solutions in some cases, but these solutions sometimes fail semantic validation, resulting in a net-zero trade.

The dominant bottleneck is the 6^K exponential scaling of validation cost with cut count, which is independent of strategy selection. Circuits that inherently require many cuts cannot be rescued by any cutting approach within the current validation framework. The LLM succeeds primarily on structurally simple circuits (ghz, bv, dj, qnn) where deterministic methods already work well, and struggles on the deeper, more entangled circuits where intelligent cutting would be most valuable. This is corroborated by sampling overhead: the LLM’s direct passes achieve a median overhead of 9.0 (one cut), matching the baseline, while fallback passes show 81.0 (two cuts), confirming that the LLM succeeds primarily on the easiest cases. This pattern, combined with the $\sim 74\%$ reroute rate, suggests that the current LLM’s circuit-analysis capability is insufficient for the harder instances in the corpus.

Despite these limitations, the results are encouraging as an architectural proof of concept. The LLM never degrades performance relative to the baseline; the pipeline’s reproducibility is high (97–99%); and the failure mode analysis points to a concrete path for improvement: the LLM finds lower-cut solutions that fail semantically, suggesting that improving the LLM’s understanding of cut validity—rather than cut finding per se—could yield genuine gains.

VI. THREATS TO VALIDITY

We organize threats following the standard classification into internal, external, and construct validity.

A. Internal Validity

The mandatory `llm_custom_cut` first-attempt policy conflates the quality of the LLM’s own cuts with the routing perturbation its attempt induces on downstream strategies. A controlled ablation (e.g., a random-perturbation baseline) would be needed to isolate these effects; the strategy attribution analysis in Section V-I partially addresses this, but a full ablation remains future work.

The cost-model feasibility gate can diverge from actual cut counts, producing spurious `VALIDATION_INFEASIBLE` outcomes that affect the two configurations asymmetrically depending on which strategy is attempted first.

Finally, with only three seeded trials and 2 discordant pairs out of 450, the experiment is substantially underpowered for small-to-moderate effect sizes. A pre-registered equivalence test with additional replicates would be needed to affirmatively conclude that the LLM adds no benefit, as opposed to the current interpretation that no benefit was detected.

B. External Validity

The benchmark corpus comprises 150 circuits drawn from 15 MQT Bench families at 3–12 qubits. This is modest in both scale and qubit range: production circuits of interest for circuit cutting typically exceed the qubit counts testable here, and the structured, textbook nature of MQT Bench families (e.g., GHZ, Deutsch–Jozsa, BV) may not reflect the irregular gate topologies encountered in application-level algorithms. Consequently, the pass rates and overhead figures reported should be interpreted as indicative rather than definitive, and generalization to larger or more heterogeneous circuit sets remains to be demonstrated.

Additionally, the evaluation is restricted to OpenQASM 2.0 gate-based circuits and does not cover OpenQASM 3.0 features or non-Qiskit frameworks.

C. Construct Validity

The primary metric—genuine pass rate—captures whether a cutting decomposition produces expectation values within tolerance of the original circuit, but it does not reward *efficiency*. A decomposition that passes with sampling overhead 81 and one that passes with overhead 6,561 are scored identically, despite an $81\times$ difference in classical post-processing cost. The overhead and timing analyses in Section V supplement the pass rate, but a composite metric balancing correctness and efficiency would provide a more complete picture.

The unified tolerance formula ($\epsilon_{\text{base}} + z \sigma_{\text{shot}}$) adapts to cut count through the coefficient-aware shot-noise term, but the choice of ϵ_{base} and z remains a researcher degree of freedom. Tightening or loosening these parameters would shift the boundary between `PASS` and `FAIL_EQUIVALENCE`, altering reported pass rates for both configurations.

Finally, equivalence validation relies on exact statevector simulation, which imposes a hard qubit ceiling. Circuits exceeding this ceiling are classified as `SKIPPED_TOO_LARGE` and excluded entirely, introducing a survivorship bias toward smaller circuits in the reported results.

VII. CONCLUSION AND FUTURE WORK

This paper asked whether an off-the-shelf LLM, embedded as an orchestrator in a multi-strategy quantum circuit-cutting pipeline, can improve upon deterministic baselines. The answer, on the MQT Bench corpus at 3–12 qubits, is no.

GRACE achieved a mean genuine pass rate of 45.6% compared to 45.1% for a deterministic-only baseline—a difference of +0.7 circuits that is not statistically significant. The

LLM’s first-attempt precision was 26.2%, with 74% of circuits recovered by deterministic fallbacks. Across all trials, 0–1 circuits per seed passed exclusively in the LLM configuration, and none through the LLM’s own cuts. The LLM added a 1.4– $3\times$ per-circuit timing overhead while producing no measurable quality gain.

We frame this as a constructive negative result for three reasons. First, the experiment demonstrates that LLM integration is *architecturally feasible*: the pipeline maintains statistical parity with the baseline, high reproducibility (97–99%), and never degrades output quality. This establishes the engineering foundation for future work without the risk that the LLM component is harmful. Second, the failure mode analysis reveals a concrete signal: the LLM does find lower-cut solutions on some circuits, but these solutions subsequently fail semantic validation, indicating that the gap lies in cut *validity* understanding rather than cut *discovery*. Third, the dominant bottleneck—exponential validation cost scaling as 6^K —is independent of strategy selection, meaning that no cutting strategy, however intelligent, can rescue circuits requiring many cuts under the current validation framework.

Several directions for future work emerge from these findings:

- **Improving LLM first-attempt precision.** The 26% success rate indicates that the off-the-shelf LLM lacks sufficient understanding of circuit-cutting constraints. Fine-tuning or few-shot prompting with validated cutting examples could improve precision and reduce the rerouting overhead that currently dominates pipeline execution time.
- **Adaptive LLM engagement.** The current mandatory `llm_custom_cut` first-attempt policy applies uniformly regardless of circuit characteristics. A selective policy that invokes the LLM only when deterministic strategies fail—or when circuit metrics predict the LLM is likely to contribute—could reduce the $\sim 74\%$ reroute rate and its associated overhead.
- **Controlled routing ablation.** The conflation of the LLM’s direct cutting quality with its indirect routing perturbation limits causal claims. A random-perturbation baseline that introduces a dummy first attempt would isolate whether any routing effects require LLM reasoning or merely benefit from exploration of the strategy space.
- **Addressing the validation bottleneck.** The 6^K exponential scaling of validation cost is the dominant barrier to higher pass rates. Approximate validation methods, sampling-based equivalence checks, or hardware-efficient reconstruction schemes could extend the reach of circuit cutting to higher cut counts.
- **Expanded benchmarking.** Extending evaluation to larger qubit counts, application-level algorithms with irregular gate topologies, and circuits expressed in OpenQASM 3.0 would test whether the LLM’s contribution grows in regimes where deterministic heuristics degrade.
- **Composite evaluation metrics.** The current genuine pass rate treats all successful decompositions equally

regardless of sampling overhead. A metric that jointly rewards correctness and penalizes overhead would better capture practical utility.

The broader lesson is that circuit cutting at this scale is bottlenecked by validation cost, not by cut-finding intelligence. Until the 6^K barrier is relaxed—through approximate validation, hardware execution, or mathematical advances in QPD overhead bounds—the space in which any cutting strategy can demonstrate superiority is narrow. GRACE provides the architecture to test whether stronger models, targeted fine-tuning, or adaptive engagement policies can exploit that space once it opens.

VIII. DATA AND SOURCE CODE AVAILABILITY

<https://github.com/qas-lab/Generative-Resilient-Automatic-Circuit-Cutting-Engine-GRACE->

IX. ACKNOWLEDGMENT

This material is based upon work supported by the U.S. National Science Foundation (NSF) under Grant No. 2349452. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

In preparing this work, we used generative AI models and tools, including OpenAI’s GPT-5 model through ChatGPT, to assist with generating and revising content, including code, text, figures, etc.

REFERENCES

- [Amazon Web Services(2025)] Amazon Web Services. 2025. Amazon announces Ocelot quantum chip. <https://www.amazon.science/blog/amazon-announces-ocelot-quantum-chip>. Accessed: 2026-02-24.
- [Brandhofer et al.(2023)] Sebastian Brandhofer, Ilia Polian, and Kevin Krulich. 2023. Optimal Partitioning of Quantum Circuits using Gate Cuts and Wire Cuts. arXiv:2308.09567 [quant-ph]
- [d’Aloisio et al.(2024)] Giordano d’Aloisio, Sophie Fortz, Carol Hanna, Daniel Fortunato, Avner Bensoussan, Eñaut Mendiluze Usandizaga, and Federica Sarro. 2024. Exploring llm-driven explanations for quantum algorithms. In *Proceedings of the 18th ACM/IEEE international symposium on empirical software engineering and measurement*. 475–481.
- [Google Quantum AI(2024)] Google Quantum AI. 2024. Meet Willow, our state-of-the-art quantum chip. <https://blog.google/innovation-and-ai/technology/research/google-willow-quantum-chip/>. Accessed: 2026-02-24.
- [Hastings et al.(2015)] Matthew B. Hastings, Dave Wecker, Bela Bauer, and Matthias Troyer. 2015. Improving quantum algorithms for quantum chemistry. *Quantum Info. Comput.* 15, 1–2 (Jan. 2015), 1–21.
- [IBM(2025)] IBM. 2025. IBM Quantum Computing — Hardware and roadmap. <https://www.ibm.com/quantum/hardware>. Accessed: 2026-02-24.
- [Peng et al.(2020)] Tianyi Peng, Aram W. Harrow, Maris Ozols, and Xiaodi Wu. 2020. Simulating Large Quantum Circuits on a Small Quantum Computer. *Physical Review Letters* 125, 15 (Oct. 2020). <https://doi.org/10.1103/physrevlett.125.150504>doi:10.1103/physrevlett.125.150504
- [Sadhu et al.(2025)] Abhishek Sadhu, Aritra Sarkar, and Akash Kundu. 2025. CutQAS: Topology-aware quantum circuit cutting via reinforcement learning. *arXiv preprint arXiv:2504.04167* (2025).
- [Schmitt et al.(2025)] Lukas Schmitt, Christophe Piveteau, and David Suter. 2025. Cutting circuits with multiple two-qubit unitaries. *Quantum* 9 (2025), 1634. <https://doi.org/10.22331/q-2025-02-18-1634>doi:10.22331/q-2025-02-18-1634 arXiv:2312.11638.
- [Shehzad et al.(2024)] Ibrahim Shehzad, Edwin Pednault, James R Garrison, Caleb Johnson, Bryce Fuller, and Jennifer R Glick. 2024. Automated cut finding and circuit knitting on large quantum circuits. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 2. IEEE, 406–407.
- [Sinha et al.(2025)] Pranav Sinha, Sumit Kumar Jha, and Sunny Raj. 2025. Circuit Partitioning Using Large Language Models for Quantum Compilation and Simulations. arXiv:2505.07711 [cs.ET] <https://arxiv.org/abs/2505.07711>
- [Ugwuishi et al.(2020)] CH Ugwuishi, UE Orji, CI Ugwu, and CN Asogwa. 2020. An overview of quantum cryptography and shor’s algorithm. *Int. J. Adv. Trends Comput. Sci. Eng* 9, 5 (2020).